

Threaded Programming

Lecture 4: Work sharing directives

Work sharing directives

- Directives which appear inside a parallel region and indicate how work should be shared out between threads
 - Parallel loops
 - Single directive
 - Masked directive

Parallel loops

- Loops are the most common source of parallelism in most codes. Parallel loop constructs are therefore very important!
- A worksharing for loop divides up the iterations of the loop between threads.
- The **for** directive appears inside a parallel region and indicates that the work should be shared out between threads, instead of replicated
- There is a synchronisation point at the end of the loop: all threads must finish their iterations before any thread can proceed.

Parallel for loops (cont)

Syntax:

C/C++:

```
#pragma omp for [clauses]  
    for loop
```

Note: no curly braces around the for loop!

Restrictions in C/C++

- Because the for loop in C is a general while loop, there are restrictions on the form it can take.
- It has to have determinable trip count - it must be of the form:
`for (var = a; var logical-op b; incr-exp)`

where *logical-op* is one of `<`, `<=`, `>`, `>=`

and *incr-exp* is `var = var +/- incr` or semantic equivalents such as `var++`.

Also cannot modify `var` within the loop body.

Parallel for loop (example)

```
#pragma omp parallel
{
    #pragma omp for
    for (int i=0;i<n;i++){
        b[i] = (a[i]*a[i-1])*0.5;
    }
}
```

Parallel for directive

- This construct is so common that there is a shorthand form which combines a parallel region and a worksharing for loop:

C/C++:

```
#pragma omp parallel for [clauses]  
    for loop
```

Parallel for (example)

```
#pragma omp parallel for  
for (int i=0;i<n;i++){  
    b[i] = (a[i]*a[i-1])*0.5;  
}
```


Clauses

- `for` directive can take `private` , `firstprivate` and `reduction` clauses which refer to the scope of the loop.
- Note that the parallel loop index variable is private by default
- `parallel for` directive can take all clauses available for `parallel` directive.
- **Beware!** `parallel for` is not the same as `for` or the same as `parallel`

Parallel for loops (cont)

- With no additional clauses, the **for** directive will partition the iterations as equally as possible between the threads.
- However, this is implementation dependent, and there is still some ambiguity:

e.g. 7 iterations, 3 threads. Could partition as $3+3+1$ or $3+2+2$

schedule clause

- The **schedule** clause gives a variety of options for specifying which loops iterations are executed by which thread.

- Syntax:

C/C++: **schedule** (*kind*[, *chunksize*])

where *kind* is one of

static, **dynamic**, **guided**, **auto** or **runtime**

and *chunksize* is an integer expression with positive value.

- e.g. **#pragma omp for schedule(dynamic,4)**

static schedule

- With no *chunksize* specified, the iteration space is divided into (approximately) equal chunks, and one chunk is assigned to each thread in order (**block** schedule).
- If *chunksize* is specified, the iteration space is divided into chunks, each of *chunksize* iterations, and the chunks are assigned cyclically to each thread in order (**block cyclic** schedule)

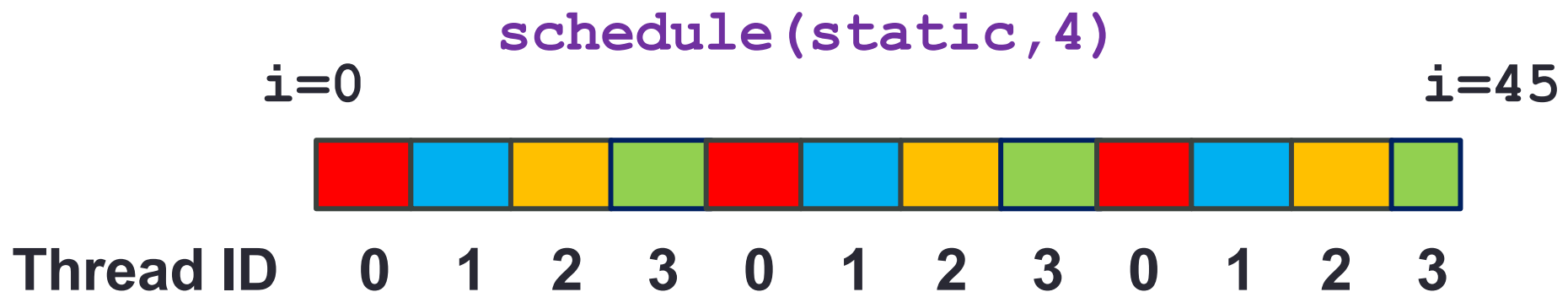
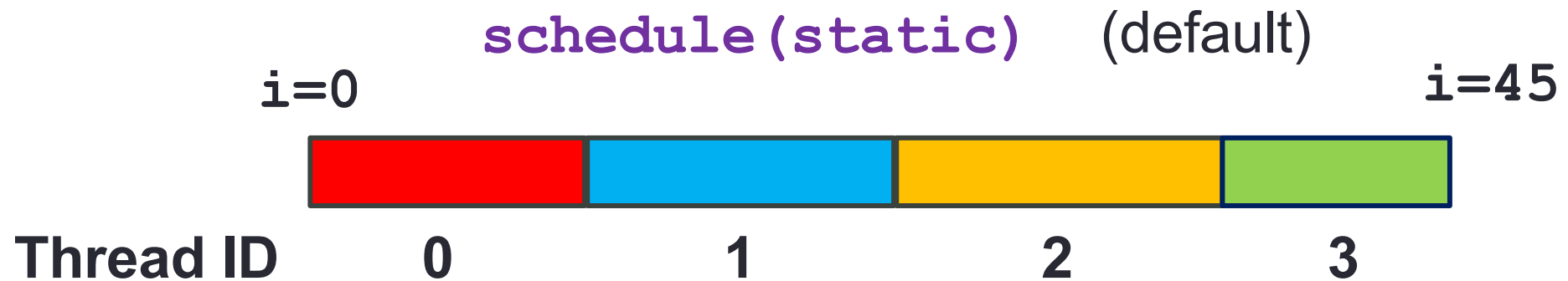
Schedule clause example

Consider this parallel loop:

```
#pragma omp parallel for  
for (int i=0;i<45;i++){  
    ...  
}
```

running on 4 threads...

static schedule



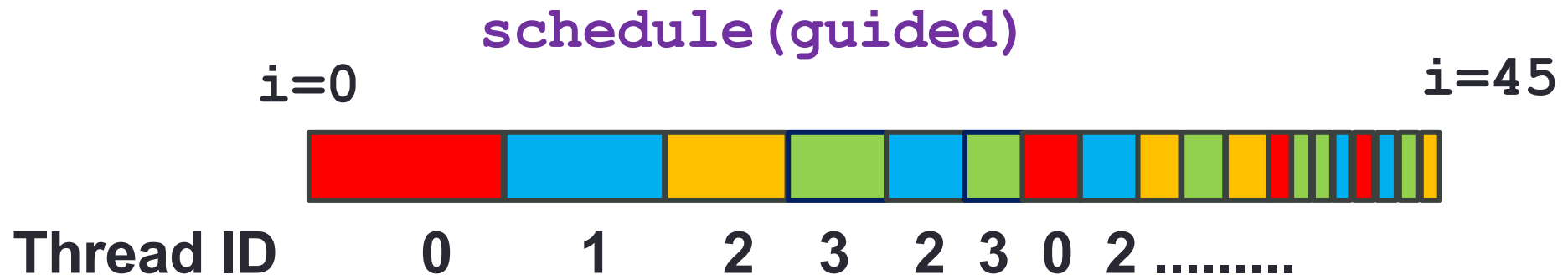
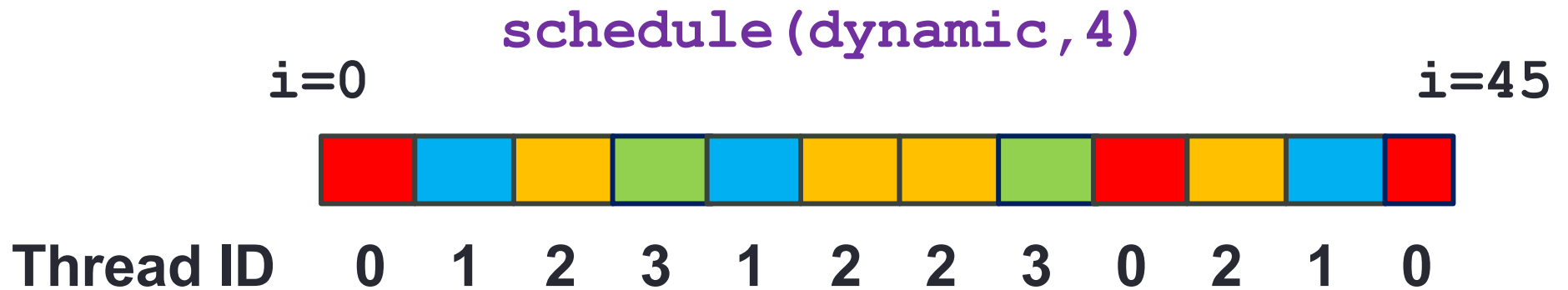
dynamic schedule

- **dynamic** schedule divides the iteration space up into chunks of size *chunksize* and assigns them to threads on a first-come-first-served basis.
- i.e. as a thread finish a chunk, it is assigned a new chunk.
- When no *chunksize* is specified, it defaults to 1.
- Chunks do not have to be scheduled in their logical sequential order (though it is possible to enforce this if desired).

guided schedule

- **guided** schedule is similar to **dynamic**, but the chunks start off large and get smaller exponentially.
- The size of the next chunk is proportional to the number of remaining iterations divided by the number of threads.
- The *chunksize* specifies the minimum size of the chunks.
- When no *chunksize* is specified it defaults to 1.

dynamic and guided schedules



auto and runtime schedules

- `schedule(auto)` lets the runtime have full freedom to choose its own assignment of iterations to threads.
- `schedule(runtime)` allows the schedule to be set using the environment variable `OMP_SCHEDULE`
 - e.g. `export OMP_SCHEDULE="dynamic,1"`
- Convenient for experimenting with schedules and chunk sizes without having to recompile.

Choosing a schedule

When to use which schedule?

- **static** usually best for load balanced loops - least overhead.
- **static,n** good for loops with mild or smooth load imbalance (i.e if the load on iteration i is not very different from that on $i+1$), but can induce overheads for small chunksizes.
- **dynamic** useful if iterations have widely varying loads, but ruins data locality, and has high overheads for small chunksizes
- **guided** has less overhead than **dynamic**, but beware of loops where the first iterations are the most expensive!
- **auto** allows compiler-specific options

single directive

- Indicates that a block of code is to be executed by a single thread only.
- The first thread to reach the **single** directive will execute the block
- There is a synchronisation point at the end of the block: all the other threads wait until block has been executed.

single directive (cont)

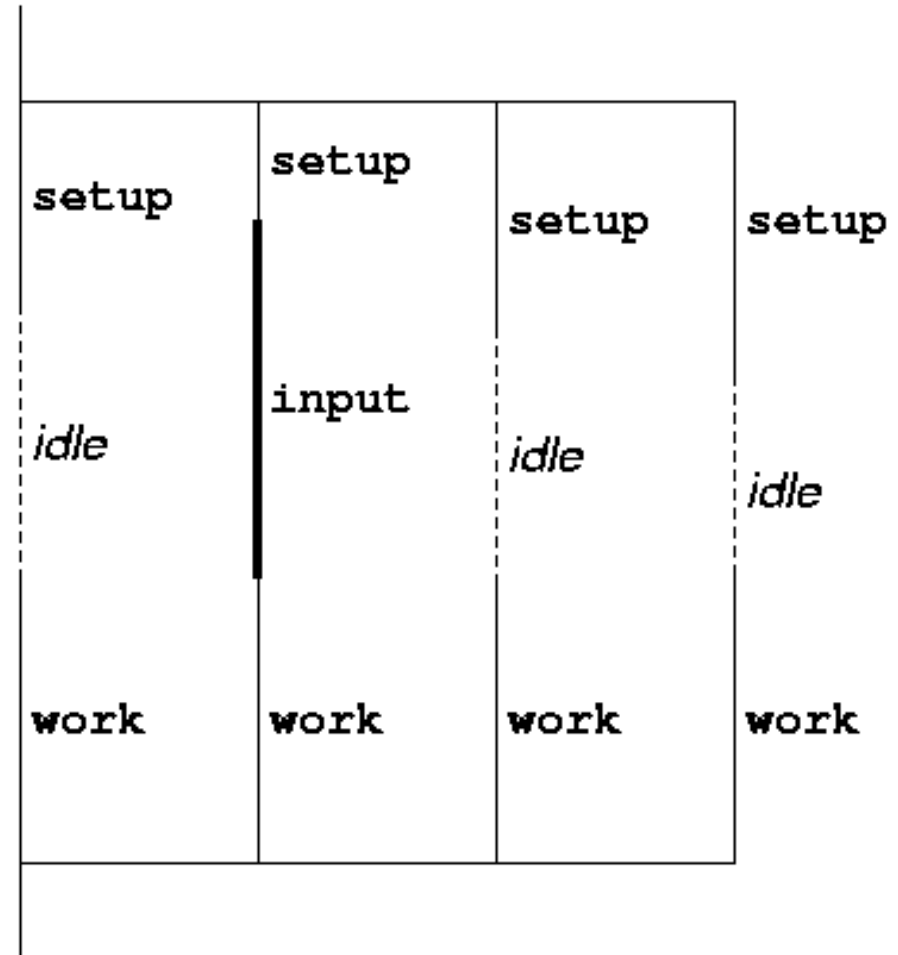
C/C++:

```
#pragma omp single [clauses]  
    structured block
```

single directive (cont)

Example:

```
#pragma omp parallel
{
    setup(x);
#pragma omp single
{
    input(y);
}
    work(x,y);
}
```



Reductions on worksharing for loops

- The result of a reduction in a loop may be required later in the parallel region.
- Need to be careful with initialising the reduction variable

```
#pragma omp parallel shared(sum, b)
{
    sum = 0.0;
#pragma omp for reduction(+:sum)
    for(i=0;i<n;i++) {
        sum += b[i];
    }
    .... = sum;
}
```

Incorrect!

Race between the initialisation of **sum** and the updates to it at the end of the loop.

Reductions on worksharing for loops

```
#pragma omp parallel shared(sum, b)
{
#pragma omp single
{
    sum = 0.0;
}
#pragma omp for reduction(+:sum)
    for(i=0;i<n;i++) {
        sum += b[i];
    }
.... = sum;
}
```

Correct!
Only one thread initialises **sum**
and there is an implicit barrier at
the end of the **single**

nowait clause

- The implicit barrier synchronization at the end of worksharing directive (**for** or **single**) can be removed by adding a **nowait** clause.
 - Use with care! Easy to introduce race conditions...

C/C++:

```
#pragma omp for nowait  
for loop
```

masked directive

- Indicates that a block of code should be executed by the primary thread (thread 0) only.
- There is no synchronisation at the end of the block: other threads skip the block and continue executing.
 - Note: different from **single** in this respect!

masked directive (cont)

Syntax:

C/C++:

```
#pragma omp masked  
    structured block
```

Note: in earlier versions of OpenMP **masked** was called **master**
Some older versions of compilers may not recognise the new form,
and many codes still use **master**

Exercise

- Redo the Mandelbrot example using a worksharing **for** directive.

Fortran syntax

Worksharing loop:

```
!$OMP DO [clauses]  
do loop  
[ !$OMP END DO ]
```

Combined directive:

```
!$OMP PARALLEL DO [clauses]  
do loop  
[ !$OMP END PARALLEL DO ]
```

Schedule clause:

```
SCHEDULE (kind[, chunksize])
```

Fortran syntax

Single:

```
! $OMP SINGLE [clauses]  
    block  
! $OMP END SINGLE
```

Nowait clause:

```
! $OMP DO  
    do loop  
! $OMP END DO NOWAIT
```

Masked:

```
! $OMP MASKED  
    block  
! $OMP END MASKED
```

Fortran parallel loop examples

```
!$OMP PARALLEL
!$OMP DO
  do i=1,n
    b(i) = (a(i)-a(i-1))*0.5
  end do
!$OMP END DO
!$OMP END PARALLEL
```

Combined version:

```
!$OMP PARALLEL DO
  do i=1,n
    b(i) = (a(i)-a(i-1))*0.5
  end do
!$OMP END PARALLEL DO
```